

CS122A Custom Laboratory Project Report

Camera system

Haokun Yang

6/10/2026

<https://youtu.be/cr1FI0NDv3g>

Introduction:

This project is a camera system that takes live video from an OV7670 camera, processes the image on an FPGA, displays the image on the 4.3 tft lcd screen, and sends the captured frame to a laptop using the wifi function on the raspberry pi pico 2w. We have multiple image modes including the original camera image, grayscale, an inverted mode, and a somewhat studio ghibli image mode.

Some parts were not fully completed. The cartoon filter is only an approximation and it's far from a true AI studio ghibli. The system also still shows some image artifacts like noise and dots. At first I planned to use the ultrasonic sensor (HC-SR04) connected to the pico to track the distance of an object and if the object got too close then it would trigger the fpga using SPI for it to take a picture and send it to us and we will send it to the laptop, but right now instead of an ultrasonic sensor, I used a button in place, but the button is still connected to the pico and use the same SPI protocol. So you can say this is maybe half implemented. I also didn't fully utilize the 480x272 tft display because my camera had 640x480 but it was too big for the screen, so I used a downscaled version of 320x240 which only takes some part of the screen but I wanted to do LVGL with touchscreen so that I can use the screen that isn't used by the camera as a button GUI so that we can use touchscreen to click "live feed", "send to pico" and "request images".

Elements of Complexity:

- Live feed camera capture from the Ov7670
- Image processing using bitwise operation with RGB 565 16 bit color like grayscale, invert and cartoon/studio ghibli approximate filter.
- SDRAM buffering for storing image frames with clock domain crossing via the async fifo as a buffer
- LCD will also read from the SDRAM to display on the 4.3 tft display
- SPI uses both mosi(pico requesting the image) and miso(we use this to send the image over to pico) and mux chooses who gets the SDRAM currently because we only have one SDRAM interface.
- WiFi/TCP to transfer the stored frame data to the laptop and then the python script will receive and piece the data back together.

Not fully implemented

- Currently we only have 4 image modes controlled by a button.
- Trigger from pico, the MOSI spi is done but the ultrasonic sensor is replaced by a button.
- I didn't fully utilize the 480x272 tft display.

Not implemented:

- I also wanted to do an LVGL with touchscreen for button GUI.

User Guides:

The user power source to power the FPGA and the Pico via usb-a and we have three buttons that are connected to this project. The camera is always running and the moment you plug it the camera will run and in a waiting state that is waiting for one of the button to be pressed and only when it's pressed then it would be a live feed if its released, then the image would show the moment when you release the button on the LCD. If you want to send that image, that is the moment you released it, to the Pico, you will press the button that is connected to the Pico to send an instruction via SPI to the FPGA. The FPGA can send all of the image data from the SDRAM to the Pico via the SPI MISO, and then it will be stored in a buffer, which is an array on the Pico. The Pico will then take that and send it over to the laptop via Wi-Fi. The third button, it's connected to an FPGA. It's a mode change button that cycles through the four mode image processing/filtering mode that I have programmed, first one is original, second one is grayscale, third one is cartoon-ish and the fourth one is inverted color. So you basically just press the live feed button so you can see the color change live, and then you release the button at whichever mode you want and whichever picture you want to take or you want to point a camera at. You will see it on the LCD, and you will be paused at that image with that color mode, and then you can send it to Pico, and then Pico will send it to your laptop.

Hardware Components Used:

- OV7670 Camera module
- IceSugar Pro FPGA
- Raspberry Pi Pico 2W
- 4.3 tft lcd display
- Debug probe
- My laptop
- 3 buttons
- Wires and resistors

Software Libraries Used:

- Pico SDK: Used for the Raspberry Pi Pico 2W, GPIO, SPI and timer.
- CYW43 Wi-Fi library which is used for the pico to connect to wifi
- lwIP TCP library, which is used to create a TCP client and send the image in bytes to the laptop.
- Python socket library which is used in the laptop receiver to listen for incoming TCP image bytes.
- Python struct library which is used to receive bytes as 16 bit pixels RGB565.
- Pillow which is used by the receiver to convert the raw RGB565 frame into a png image.

Protocols Used:

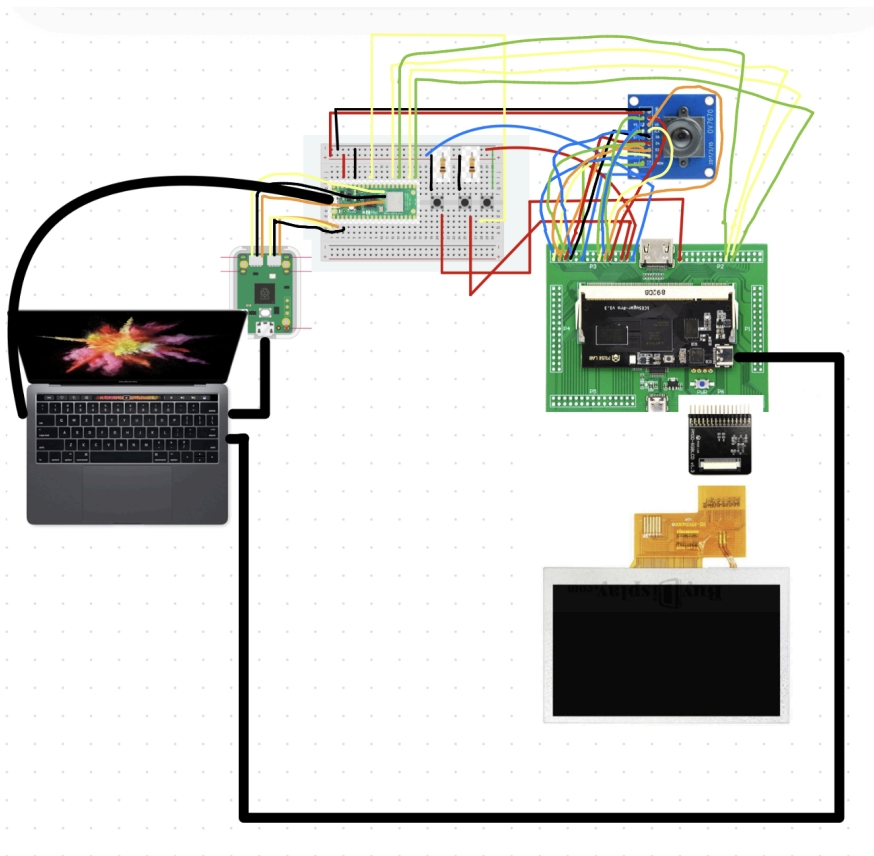
- SCCB/I2C-ish protocols which are used to configure the OV7670 camera registers for things like the color format or the image scales and many more.
- SPI which is used between the PICO 2W and the FPGA so that the FPGA can send the image data to the pico when pico requests it and pico would send it to the laptop afterwards.
- The OV7670 camera also had a parallel 8 bit sending pixel and we combined it to 16 bit. It also has a vsync that tells you the end and begin of a frame and href for telling us the end of a scan line.
- TCP and IP with wifi which is used by the pico to send the image on the buffer to the laptop.
- I also used RGB 565 if that counts as a protocol.

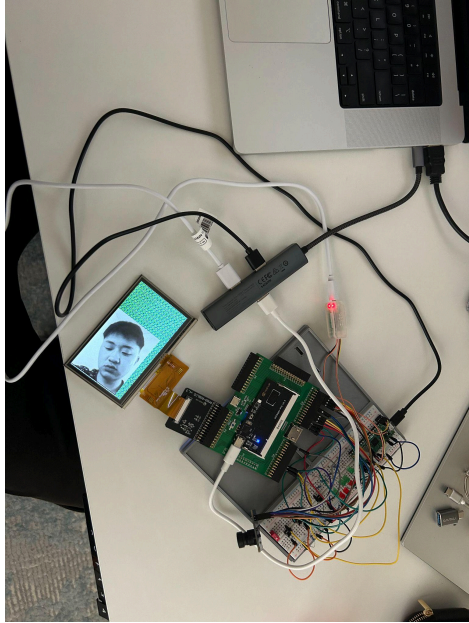
Requirements:

- Icesugar Pro FPGA: I use FPGA as the device that processes the image that we capture from the camera, and also we add image processing, which is like a filter that manipulates the value of the pixel at individual addresses to give us the desired outcome. We use an RGB565 format, and all of our image data is stored on the SDRAM, which it's used by the LCD display and also the SPI that goes to the Pico to send the image to the laptop and we manage both of this using a mux.
- RGB LCD: I use the LCD as the display for the image that is captured by the camera. The LCD reads from the SDRAM on the FPGA because the camera stores all the pixels in the SDRAM via a buffer because they're running on different clocks. It shows the color mode as a normal image, gray scale, cartoon style filtering, and an inverted image. We're able to see a live feed when the button is pressed through the LCD. We can change the mode in real time as well.

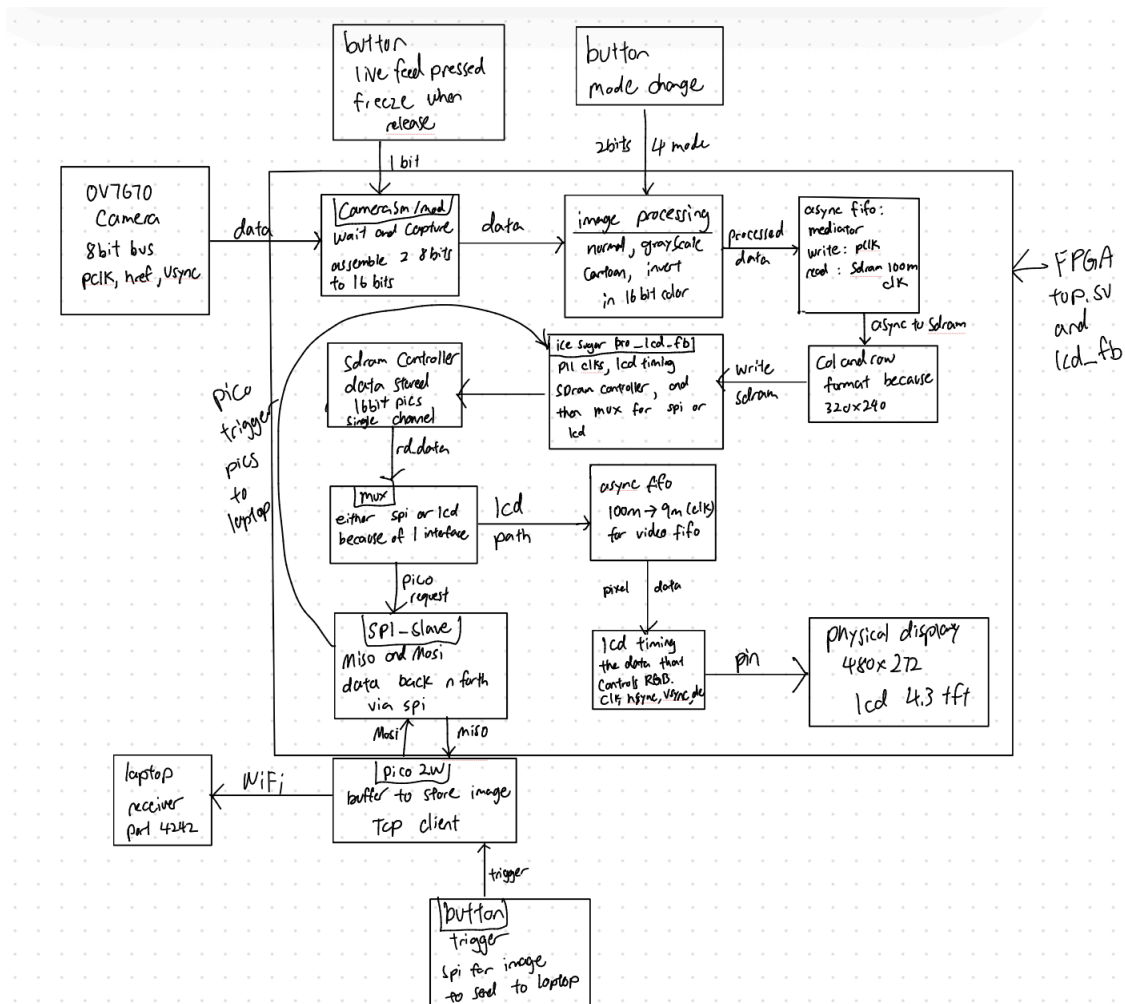
- Raspberry Pi Pico: I use the Pi Pico as a device that transfers data that we get from FPGA to the laptop. The pico will first send a command through the button that is connected to the pico via SPI to the FPGA to tell the FPGA we want the data from your SDRAM. The FPGA will see that command and then send whatever it has in the SDRAM via SPI miso back to the pico, so the pico acts like a master and the FPGA is the slave. And then the pico we'll get the image, and it will be on the buffer on the Pico, which is the SRAM. Through that, we send it to the laptop via TCP client.
- Event Driven execution: So I used event-driven execution through the button action. On the FPGA, we use the button to change the image processing mode. On the Pico side, we press the button so that it will send a trigger through SPI that tells the FPGA to send us the image data. And then we start a Wi-Fi transfer to the laptop. And the TCP also uses lwIP callbacks to send the image data.

Wiring Diagram:





Design Diagram:



AI Usage:

I used AI to write my WiFi code, the python file receiver script and the code that exists on the pico side for the wifi related part are all written by ai. I also used ai to help me debug syntax errors, logic errors that I was stuck on, and the cameraROM to play with the configuration because of the pink/green tint bug, but then I discovered that there is no “R7” pin on the FPGA, its supposed to be “P7” so the camera ROM register is modified a little bit. AI also helped with debugging the bitwise operation and the AI did the cartoon filter a studio ghibli attempt. AI also helped me understand the SPI slave code and camera code I got from github and debug for syncing the sck and the pclk. I used AI as a debug assistant and as a tutor

Acknowledgements:

All the sources that I used or referenced.

My camera configuration with all the 75 register is from this repo:

https://github.com/westonb/OV7670-Verilog/blob/master/src/OV7670_config_rom.v

https://github.com/amsacks/OV7670-camera/blob/main/rtl/cam_rom.v also looked at this repo.

<https://gemini.google.com/share/b6bfe966d9c8>

<https://claude.ai/share/793738dc-87d9-4a8a-ae1e-fac713c591c3> Testbench

<https://claude.ai/share/895a7ec6-6c3c-4abd-a491-dbea3f32a2c8>

<https://claude.ai/share/bb518139-a431-44e4-a457-8b87a3842b43>

<https://claude.ai/share/1fe139c0-6607-4b11-be67-19a338aec0f6>

<https://www.fpga4student.com/2018/08/basys-3-fpga-ov7670-camera.html>

<https://github.com/yosyshq/oss-cad-suite-build>

<https://github.com/UCR-CS122A>

<https://www.eevblog.com/forum/projects/ov7670-color-settings/>

https://sourcevu.sysprogs.com/rp2040/picosdk/symbols/spi_read_blocking#dst

<https://stackoverflow.com/questions/58449462/rgb565-to-grayscale>

https://github.com/nandland/spi-master/blob/master/Verilog/source/SPI_Master.v

https://github.com/nandland/spi-slave/blob/master/Verilog/source/SPI_Slave.v

<https://www.fpga4fun.com/SPI2.html>

<https://forums.raspberrypi.com/viewtopic.php?t=352822>

<https://www.youtube.com/watch?v=TR0Pw89EuGk&t=206s>

<https://www.youtube.com/watch?v=0nVNwozXslc>

https://www.google.com/search?q=image+processing+on+fpga+bitwise+operation&sca_esv=4a455b0f3ad2390b&sxsrf=ANbL-n7KK-oc9GbGjVYEQ-MxwMLveQmu0w%3A1781072610527&ei=4gIpavD0H5m5kPIP8-jw2QQ&ved=0ahUKEwjw7ZPChPyUAxWZHEQIHXM0PEsQ4dUDCBA&uact=5&oq=image+processing+on+fpga+bitwise+operation&gs_lp=Egxnd3Mtd2l6LX

[NlcnAiKmltYWdlIHByb2Nlc3Npbmcgb24gZnBnYSBiaXR3aXNlIG9wZXJhdGlvbjIKEAAYR
xjWBBiwAzIKEAAYRxjWBBiwAzIKEAAYRxjWBBiwAzIKEAAYRxjWBBiwAzIKEAAYR
xjWBBiwAzIKEAAYRxjWBBiwAzIKEAAYRxjWBBiwAzIKEAAYRxjWBBiwA0iYKlAAW
ABwAngAkAEAmAE7oAE7qgEBMbgBA8gBAJgCAqACCZgDAIgGAZAGCJIHATKgB8AF
sgcAuAcAwgcFMC4xLjHIBwaACAE&sclient=gws-wiz-serp](#)